EE-170L Computer Programming Lab

Laboratory Report — No. 04

Switch Statement, break and continue in C++

Student Name	Zawar Shah
Registration No.	BF25NWELE0677
Course	EE-170L – Computer Programming Lab
Lab No.	04
Topic	Switch Statement, break Statement, continue Statement
Semester	Spring 2026
Institution	UET Peshawar

1. Lab Objectives

By the end of this lab session, students will be able to:

- Understand and use the switch statement as a multi-way selection tool in C++.
- Apply the break statement to terminate case blocks and prevent fall-through.
- Understand the role of the default section in handling unmatched cases.
- Use the continue statement inside loops to skip iterations.
- Differentiate clearly between break and continue and when to use each.
- Develop practical menu-driven programs using switch.

2. Theoretical Background

2.1 The switch Statement

The switch statement is used to select one block of code from multiple options based on the value of a single variable or expression. It is commonly used as a cleaner alternative to a long if-else-if chain when comparing one variable against several constant values.

Key characteristics of the switch statement:

- The switch expression must evaluate to an integer or character constant.
- Each case label must have a unique constant value — duplicate cases are not allowed.
- It falls under the category of selection statements, alongside if and else.
- It is called a multiple-selection statement because it can branch to many different actions.

Syntax:

```
switch (expression)
{
```

```
    case value1:
        // code block 1
        break;

    case value2:
        // code block 2
        break;

    default:
        // default code block (optional)
}
```

2.2 The break Statement

The break statement is used inside a switch block to terminate a case and prevent fall-through. Without break, execution continues into the next case regardless of whether it matches.

- Case statements mark where execution begins inside the switch block.
- break statements mark where execution ends for that case.
- Without break, all remaining cases execute until the closing brace or a break is found.
- break is optional — intentional fall-through is valid C++ but should be commented clearly.

2.3 The default Section

The default section is an optional part of a switch that executes when no case value matches the switch expression. It acts as a catch-all and is typically placed at the end of the switch block.

- The default section is not required; the program will run fine without it.
- If included, it handles all unmatched inputs and prevents silent failures.

2.4 The continue Statement

The continue statement is used inside loops (for, while, do-while) to skip the rest of the current iteration and jump directly to the next iteration. Unlike break, it does not exit the loop entirely.

- **break:** Exits the loop or switch block immediately.
- **continue:** Skips the current loop iteration and moves to the next one.

3. Example Program — Simple Calculator Using switch

The following program demonstrates how switch is used to build a simple four-operation calculator. The operator character entered by the user determines which arithmetic operation is performed:

```
#include <iostream>
using namespace std;

int main()
{
    int  num1, num2;
    char op;

    cout << "Enter two numbers: ";
    cin  >> num1 >> num2;
    cout << "Enter operator (+, -, *, /): ";
```

```
    cin  >> op;

    switch (op)
    {
        case '+':
            cout << "Result = " << num1 + num2;
            break;

        case '-':
            cout << "Result = " << num1 - num2;
            break;

        case '*':
            cout << "Result = " << num1 * num2;
            break;

        case '/':
            cout << "Result = " << num1 / num2;
            break;

        default:
            cout << "Invalid operator!";
    }
    return 0;
}
```

4. Lab Tasks & Solutions

Task 1 — Day Name from Number Using switch

Write a C++ program that prompts the user to enter a day number (1–7) and uses a switch statement to output the corresponding day name (1 = Monday, 7 = Sunday). A break statement ends each case and default handles invalid input.

```
#include <iostream>
using namespace std;

int main()
{
    int day;
    cout << "Enter day number (1-7): ";
    cin  >> day;

    switch (day)
    {
        case 1:
            cout << "Monday"    << endl; break;
        case 2:
            cout << "Tuesday"   << endl; break;
        case 3:
            cout << "Wednesday" << endl; break;
        case 4:
            cout << "Thursday"  << endl; break;
        case 5:
            cout << "Friday"    << endl; break;
        case 6:
            cout << "Saturday"  << endl; break;
        case 7:
            cout << "Sunday"    << endl; break;
        default:
            cout << "Invalid day number! Enter 1-7." << endl;
```

```
    }
    return 0;
}
```

How it works:

- The program prompts the user to enter a number between 1 and 7.
- The switch statement matches the entered number to the correct case.
- The corresponding day name is printed using cout.
- break ends each case to prevent fall-through into subsequent cases.
- The default case handles any number outside the valid range 1–7.

Task 2 — Grade Description Using switch

Write a C++ program using switch that takes a character grade (A, B, C, D, F) from the user and prints the corresponding description. Any other character should display "Invalid Grade".

```
#include <iostream>
using namespace std;

int main()
{
    char grade;
    cout << "Enter your grade (A/B/C/D/F): ";
    cin  >> grade;

    switch (grade)
    {
        case 'A': case 'a':
            cout << "Excellent"   << endl; break;
        case 'B': case 'b':
            cout << "Very Good"   << endl; break;
        case 'C': case 'c':
            cout << "Good"        << endl; break;
        case 'D': case 'd':
            cout << "Pass"        << endl; break;
        case 'F': case 'f':
            cout << "Fail"        << endl; break;
        default:
            cout << "Invalid Grade" << endl;
    }
    return 0;
}
```

Grade mapping used:

Grade	Output
`A`	Excellent
`B`	Very Good
`C`	Good
`D`	Pass
`F`	Fail
`Other`	Invalid Grade

Task 3 — Unit Conversion Menu Using switch

Write a C++ program using switch that presents a conversion menu to the user. The user selects a conversion type, enters a value, and the program performs and displays the result. Invalid choices are handled by the default case.

Menu options:

- 1 → Convert Kilometres to Metres (multiply by 1000)
- 2 → Convert Metres to Centimetres (multiply by 100)
- 3 → Convert Kilograms to Grams (multiply by 1000)
- 4 → Convert Celsius to Fahrenheit (F = C × 9/5 + 32)

```
#include <iostream>
using namespace std;

int main()
{
    int    choice;
    double value, result;

    cout << "===== Unit Converter =====" << endl;
    cout << "1. Kilometres  -> Metres"    << endl;
    cout << "2. Metres      -> Centimetres" << endl;
    cout << "3. Kilograms   -> Grams"     << endl;
    cout << "4. Celsius     -> Fahrenheit" << endl;
    cout << "Enter your choice (1-4): ";
    cin  >> choice;

    switch (choice)
    {
        case 1:
            cout << "Enter distance in km: ";
            cin  >> value;
            result = value * 1000;
            cout << value << " km = " << result << " m" << endl;
            break;

        case 2:
            cout << "Enter distance in metres: ";
            cin  >> value;
            result = value * 100;
            cout << value << " m = " << result << " cm" << endl;
            break;

        case 3:
            cout << "Enter mass in kg: ";
            cin  >> value;
            result = value * 1000;
            cout << value << " kg = " << result << " g" << endl;
            break;

        case 4:
            cout << "Enter temperature in Celsius: ";
            cin  >> value;
            result = (value * 9.0 / 5.0) + 32;
            cout << value << " C = " << result << " F" << endl;
            break;

        default:
            cout << "Invalid choice! Please enter 1-4." << endl;
```

```
    }
    return 0;
}
```

5. break vs continue — Key Differences

Feature	break	continue
Purpose	Exits the loop or switch block	Skips current iteration of the loop
Used in	Loops and switch statements	Loops only (for, while, do-while)
Effect	Terminates the entire loop/switch	Moves directly to the next iteration
Remaining code	Skipped; exits the block immediately	Skipped for current iteration only

Example — continue skipping odd numbers:

```
#include <iostream>
using namespace std;

int main() {
    // Print only even numbers from 1 to 10
    for (int i = 1; i <= 10; i++) {
        if (i % 2 != 0)
            continue;   // skip odd numbers
        cout << i << " ";
    }
    return 0;
}

// Output: 2 4 6 8 10
```

6. Conclusion

This lab provided a practical understanding of program flow control in C++ using the switch, break, and continue statements. The key takeaways are:

- **switch Statement:** A clean and efficient way to handle multi-way branching based on a single variable's value, replacing lengthy if-else-if chains.
- **break Statement:** Essential within switch cases to prevent unintended fall-through; also used to exit loops early when a condition is met.
- **default Section:** An optional but recommended safety net in switch that catches all unmatched case values.
- **continue Statement:** Used inside loops to skip the remainder of the current iteration and proceed directly to the next, without exiting the loop.
- **Menu-Driven Programs:** The switch statement is ideal for building interactive menus where each option maps to a distinct code block.

Understanding these control flow tools is fundamental for writing clear, efficient, and correct C++ programs. They will be applied extensively in upcoming labs involving loops and functions.